

Intern Briefing: BIN-FSN Stockout Diagnosis UI

1. Executive Summary & Problem Statement

In Flipkart Hyperlocal dark stores, items live in labeled slots called **BINs** (e.g., F1-05-5D). Each item is identified by a unique **FSN** (Flipkart Serial Number). When a picker cannot physically find an FSN in its designated BIN, they raise an **INF (Item Not Found)** ticket. This triggers an Inventory Reconciliation Ticket (IRT), causes the customer's order to fail, and directly drops the dark store's overall **fill rate**.

While the existing S11 signal flags which FSN/BIN pairs keep failing, it lacks root-cause context. Determining the "why" manually takes an analyst 3 to 5 days across multiple dashboards. This project aims to reduce that diagnostic latency down to minutes by mapping failures to actionable categories using an automated Web UI and Graph LLM assistant.

2. Root-Cause Core Logic & Actions

The core diagnostic paradigm is simple but powerful:

Diagnostic Insight	Root Cause	Action Required
Many distinct FSNs failing in the same BIN	PHANTOM INVENTORY	Stocktake the BIN: System thinks stock is present, but it is physically misplaced, stolen, or missing.
The same FSN failing across many distinct BINs	GENUINE STOCKOUT	Replenish the FSN: The inventory is totally depleted across the entire warehouse facility.

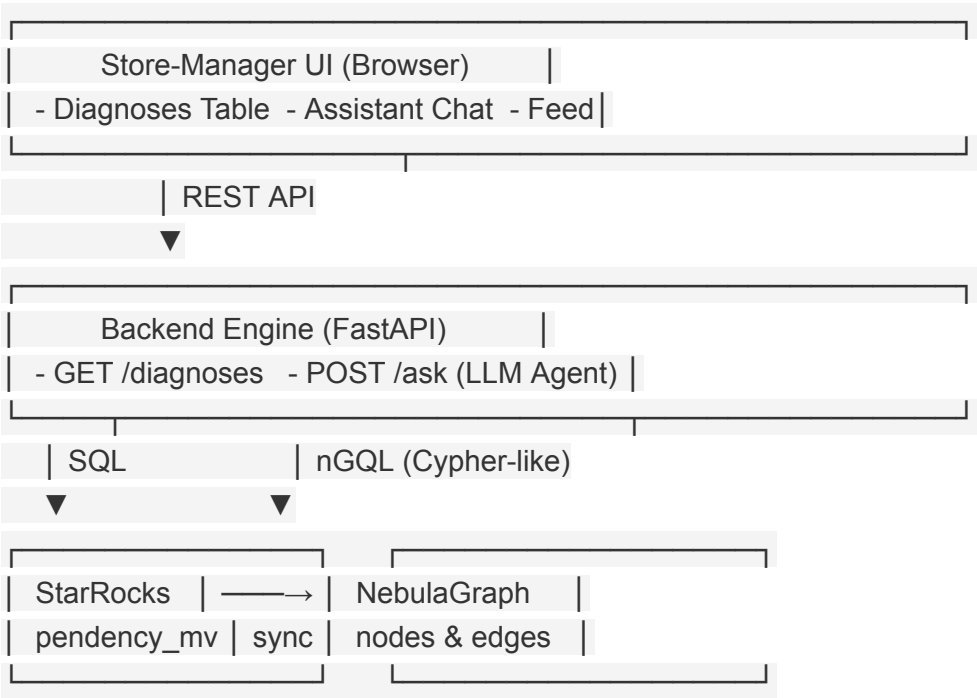
3. Product Requirements & Architecture

You are building a central web UI for store managers. They will log in directly via their browsers; no auxiliary Slack or email notifications are included in this scope.

Key UI Capabilities

- **Diagnoses Table:** A ranked list of issues mapped by (warehouse, BIN, FSN), showing the calculated verdict and an expected metric recovery projection.
- **"Ask the Assistant" Box:** An interactive LLM agent box answering plain English natural language questions (e.g., *"Why is BIN F1-05-5D failing?"* or *"How can I improve fill rate today?"*). **Crucial:** Every claim must cite underlying graph or SQL query results.
- **Feedback View:** A closed-loop tracking layout to confirm if recommendations were executed and verify whether localized failures successfully ceased.

System & Data Architecture



Tech Stack Breakdown:

- **Analytics Layer:** StarRocks (Source table: hl_customer_outbound.pendency_mv).
- **Graph Store:** NebulaGraph. Used for complex multi-hop traversals (e.g., tracing picker overlaps and shared inbound batches) that are costly/complex in traditional SQL.
- **Backend Framework:** Python (FastAPI). Exposes diagnostics APIs and orchestrates the

LLM.

- **LLM Models:** Claude Haiku 4.5 for routine routing/tagging; Claude Sonnet 4.6 for complex reasoning.

4. Source Schema & Prototyping SQL

All core source metrics live inside `hl_customer_outbound.pendency_mv`. Do not request or build any secondary materialized views.

Key Schema Columns

- `reservation_warehouse_id`: Unique identifier for the specific dark store.
- `picklist_source_location_label`: Physical slot identifier (The BIN).
- `picklist_item_fsn`: Stock keeping unit identifier (The FSN).
- `irt_ticket_id`: Non-null values signify an active INF event.
- `irt_ticket_type`: Enum tracking infraction categorization.
- `picklist_assigned_to`: Unique ID of the deployed picker.
- `order_id`: Impacted customer transaction sequence.
- `updated_at`: Timestamp.

StarRocks Validation Query

Run this diagnostic framework directly inside your StarRocks client to audit baseline verdicts against local analyst insight:

```
WITH inf AS (  
  SELECT reservation_warehouse_id AS wh,  
         picklist_source_location_label AS bin,  
         picklist_item_fsn AS fsn,  
         COUNT(*) AS failures,  
         COUNT(DISTINCT order_id) AS orders  
  FROM hl_customer_outbound.pendency_mv  
  WHERE updated_at >= NOW() - INTERVAL 1 DAY  
  AND irt_ticket_id IS NOT NULL  
  GROUP BY 1,2,3  
)  
bin_sum AS (SELECT wh, bin, COUNT(DISTINCT fsn) AS distinct_fsns FROM inf GROUP  
BY 1,2),  
fsn_sum AS (SELECT wh, fsn, COUNT(DISTINCT bin) AS distinct_bins FROM inf GROUP  
BY 1,2)
```

```

SELECT i.*, b.distinct_fsns, f.distinct_bins,
CASE
  WHEN b.distinct_fsns >= 3 AND f.distinct_bins >= 2 THEN 'DUAL'
  WHEN b.distinct_fsns >= 3 THEN 'PHANTOM'
  WHEN f.distinct_bins >= 2 THEN 'GENUINE_STOCKOUT'
  ELSE 'AMBIGUOUS'
END AS verdict
FROM inf i
JOIN bin_sum b USING (wh, bin)
JOIN fsn_sum f USING (wh, fsn)
ORDER BY orders DESC;

```

5. Core 7-Step Implementation Plan

- Profile & Audit Data:** Read hl_customer_outbound.pendency_mv via your local StarRocks client to inspect real-time row density.
- Validate intuition:** Execute the diagnostic SQL query. Manually evaluate the generated verdicts across 5 sample pilot BINs alongside store experts.
- Spin Up Graph Instance:** Stand up NebulaGraph locally using Docker. Map structural nodes (FSN, BIN, Picker, Order) and directional edges (FAILED_AT, PICKED_FROM, ASSIGNED_TO).
- Pipeline Construction:** Author a lightweight Python ETL cron script syncing StarRocks data into NebulaGraph nodes and edges at a strict 1-minute interval.
- Construct Service Backend:** Code the FastAPI core. Expose standard relational responses on GET /diagnoses and structured LLM tool execution logic on POST /ask.
- Assemble User Interfaces:** Code the frontend web app using React, grouping operations cleanly between the master grid view and the natural language agent portal.
- Launch Pilot Phase:** Deploy for a single physical dark store facility. Instruct the engine to capture tracking patterns inside a dedicated recommendation_log table.

6. Project Guardrails & Scope Boundaries

IN SCOPE (Focus Here)	OUT OF SCOPE (Do Not Build)
Querying the existing pendency_mv	Creating new StarRocks materialized

IN SCOPE (Focus Here)	OUT OF SCOPE (Do Not Build)
StarRocks layout directly. - Local Docker-based NebulaGraph setups. - React dashboard components with an embedded text-based LLM query canvas. - Closed-loop action capture in recommendation_log.	views. - Building push notification nodes, Slack hooks, or transactional emails. - Automated, unattended API execution of stocktakes. - Machine Learning forecasting models or picker behavioral coaching tracks. - Fine-tuning custom LLM weight files.

7. Immediate Team Clarifications Needed

Please clarify the following parameters with your engineering leads during your first week alignment session:

- What is the explicit string/integer irt_ticket_type enum value that strictly isolates an INF event?
- Is the column picklist_source_location_label verified to be 1:1 with the physical BIN label across all regions?
- How are operational closures (such as completed stocktakes or replenishment events) pushed back into system logs to cleanly measure feedback loops?
- What is the standard UI authentication system required for local store managers (e.g., Flipkart SSO)?
- Is there an established token spend or cost ceiling configured per warehouse per day for the LLM infrastructure?
- Should we rely on an enterprise-managed NebulaGraph cluster instance, or fully own self-hosted architecture configurations?

8. Definitive Success Metrics

- The internal diagnostic UI must remain continuously operational in a target pilot dark store for **≥ 1 week**.
- Automated backend analytical verdicts must yield **≥ 70% accuracy match rate** against historical manual validation choices recorded by operations analysts.
- The interactive assistant must answer diagnostic and "what-if" projection workflows while cleanly citing specific underlying datasets for every single claim made.
- End-to-end interface execution latency must record at **less than 10 seconds** from initial

button interaction or query submit to render.

- The system must maintain fully audit-ready evidence strings linking explicit UI suggestions directly to operations team changes and final resolution outcomes.